

Cleo.ai - Complete Platform Enhancement Prompt

Overview

Fix all critical issues and implement new features for the Cleo.ai AI Superconnector platform. This is a members-only professional networking platform connecting founders, investors, and talent in India's startup ecosystem.

CRITICAL BUG FIXES (Priority: P0)

1. Fix Privacy Page Performance Issue

Problem: Privacy page takes 9+ seconds to load **Expected:** < 2 seconds load time

Tasks:

- Profile database queries - check for N+1 queries
- Add server-side caching for static legal pages
- Implement Redis caching for repeated queries
- Add database query optimization (indexes, eager loading)
- Add CDN caching headers for static content
- Consider generating Privacy/Terms as static HTML at build time

2. Implement Mobile Navigation

Problem: No mobile-optimized navigation menu **Expected:** Hamburger menu with slide-out drawer

Implementation:

- Add hamburger icon (top-right) visible only on mobile (< 768px)
- Create slide-out drawer component (right side)
- Include: Logo, How It Works, Testimonials, Log In, Get Started CTA
- Add overlay backdrop (click to close)
- Implement focus trap for accessibility
- Add swipe-to-close gesture support
- Use Framer Motion for smooth 300ms transitions
- Maintain scroll position when menu opens

Mobile Menu Structure:

```
<MobileNav>
  <CloseButton />
  <Logo />
  <NavLinks>
    <Link to="#how-it-works">How It Works</Link>
    <Link to="#testimonials">Testimonials</Link>
    <Link to="/login">Log In</Link>
  </NavLinks>
  <CTAButton>Get Started <img alt="arrow icon" data-bbox="305 825 315 835"/></CTAButton>
</MobileNav>
```

3. Fix Broken Contact Link

Problem: Footer “Contact” link doesn’t work **Solution:** Either create /contact page OR remove the link temporarily

Contact Page Requirements:

- Contact form with fields: Name, Email, Subject, Message
- Email sends to: support@cleo.ai
- Success/error toast notifications
- FAQ section (5-10 common questions)
- Response time expectation (24-48 hours)

UI/UX ENHANCEMENTS (Priority: P1)

4. Add Loading States & Feedback System

A. Button Loading States

```
<Button
  loading={isSubmitting}
  loadingText="Creating account..."
>
  Get Started
</Button>
```

- Show spinner inside button
- Disable button during submission
- Change text to indicate action

B. Toast Notification System

```
// Success
toast.success("Account created successfully! Welcome to Cleo.")

// Error
toast.error("Something went wrong. Please try again.")

// Info
toast.info("New investor match available!")

// Warning
toast.warning("Please complete your profile for better matches.")
```

- Position: top-right desktop, top-center mobile
- Auto-dismiss: 5 seconds
- Manual close option
- Max 3 toasts visible

C. Skeleton Screens

```
// For match cards
<SkeletonMatchCard />
```

```
// For profile content
<SkeletonProfile>
  <SkeletonAvatar size="lg" />
  <SkeletonText lines={3} />
  <SkeletonBadge count={4} />
</SkeletonProfile>
```

- Use pulse animation
- Match the layout of actual content
- Show during data fetching

5. Form Validation Improvements

Signup Form Validation:

```
Full Name:
- Required
- Min 2 characters
- Error: "Please enter your full name"

Email:
- Required
- Valid email format
- Check availability (debounced 500ms)
- Error: "Please enter a valid email"
- Error: "This email is already registered"

Password:
- Required
- Min 8 characters
- At least 1 uppercase
- At least 1 number
- At least 1 special character (!@#$$%^&*)
- Show strength indicator (Weak/Fair/Strong)
- Error: "Password must be at least 8 characters"

Confirm Password:
- Must match password
- Error: "Passwords do not match"

User Type:
- Required selection
- Options: Founder, Investor, Talent
```

Real-time Validation:

- Validate on blur for each field
- Show inline error messages below fields
- Green checkmark for valid fields

- Red border for invalid fields

6. Accessibility Improvements

A. Add ARIA Landmarks

```
<header role="banner">  
<nav role="navigation">  
<main role="main">  
<footer role="contentinfo">
```

B. Focus Management

- Visible focus indicators (2px outline, offset 2px)
- Focus trap in modals
- Return focus to trigger on modal close

C. Color Contrast

- Ensure 4.5:1 ratio for all text
- Test with WebAIM contrast checker
- Fix any failing combinations

D. Skip Link

```
<a href="#main-content" class="skip-link">  
  Skip to main content  
</a>
```

7. Add Meta Tags

Theme Color for Mobile:

```
<meta name="theme-color" content="#0F0F1A">  
<meta name="apple-mobile-web-app-capable" content="yes">  
<meta name="apple-mobile-web-app-status-bar-style" content="black-translucent">
```

NEW FEATURES (Priority: P2)

8. In-App Messaging System

Database Schema:

```
-- Conversations table  
CREATE TABLE conversations (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  participant_1_id UUID NOT NULL REFERENCES users(id),  
  participant_2_id UUID NOT NULL REFERENCES users(id),  
  created_at TIMESTAMP DEFAULT NOW(),  
  last_message_at TIMESTAMP,  
  UNIQUE(participant_1_id, participant_2_id)  
);
```

```

-- Messages table
CREATE TABLE messages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  conversation_id UUID NOT NULL REFERENCES conversations(id) ON DELETE CASCADE,
  sender_id UUID NOT NULL REFERENCES users(id),
  content TEXT NOT NULL,
  attachments JSONB DEFAULT '[]',
  read_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT NOW()
);

-- Indexes for performance
CREATE INDEX idx_messages_conversation ON messages(conversation_id);
CREATE INDEX idx_messages_created_at ON messages(created_at DESC);
CREATE INDEX idx_conversations_participant ON conversations(participant_1_id, participant_2_id);
CREATE INDEX idx_conversations_last_message ON conversations(last_message_at DESC);

```

UI Components:

```

// Conversation List (Sidebar)
<ConversationList>
  {conversations.map(conv => (
    <ConversationItem
      key={conv.id}
      name={conv.otherParticipant.name}
      avatar={conv.otherParticipant.avatar}
      lastMessage={conv.lastMessage}
      unreadCount={conv.unreadCount}
      timestamp={conv.lastMessageAt}
      isOnline={conv.otherParticipant.isOnline}
      isActive={conv.id === activeConversationId}
    />
  ))}
</ConversationList>

// Chat Interface
<ChatWindow>
  <ChatHeader
    name={otherUser.name}
    role={otherUser.role}
    company={otherUser.company}
    isOnline={otherUser.isOnline}
  />
  <MessageList>
    {messages.map(msg => (
      <MessageBubble
        key={msg.id}
        content={msg.content}
        isOwn={msg.senderId === currentUser.id}
        timestamp={msg.createdAt}

```

```

        isRead={!msg.readAt}
      />
    ))}
    {isTyping && <TypingIndicator />}
  </MessageList>
  <MessageInput
    placeholder="Type a message..."
    onSend={handleSendMessage}
    attachments={true}
    templates={messageTemplates}
  />
</ChatWindow>

```

Message Templates:

```

const messageTemplates = [
  {
    id: 'pitch-intro',
    label: 'Pitch Introduction',
    content: 'Hi {{name}}, I\'m raising a {{stage}} round for {{company}} in the {{sector}} space. Would
      ↪ love to share our pitch deck and explore if there\'s a fit with your investment thesis.'
  },
  {
    id: 'follow-up',
    label: 'Follow Up',
    content: 'Hi {{name}}, following up on my previous message. Would you be open to a brief call next
      ↪ week to discuss {{company}}?'
  },
  {
    id: 'deck-share',
    label: 'Share Deck',
    content: 'Hi {{name}}, thanks for connecting! I\'ve attached our pitch deck for your review. Looking
      ↪ forward to your thoughts.'
  }
]

```

WebSocket Events:

```

// Server events
socket.on('message:new', handleNewMessage);
socket.on('message:read', handleMessageRead);
socket.on('user:typing', handleUserTyping);
socket.on('user:online', handleUserOnline);

// Client events
socket.emit('message:send', messageData);
socket.emit('message:read', messageId);
socket.emit('user:typing', { conversationId, isTyping });

```

Features:

- Real-time messaging via WebSocket

- Read receipts
- Typing indicators
- File attachments (PDF, images max 10MB)
- Message templates
- Push notifications
- Email digests for unread messages
- Block/report users

9. Advanced Search & Filters

Search Interface:

```
<SearchPage>
  <SearchBar
    placeholder="Search founders, investors, or talent..."
    value={searchQuery}
    onChange={setSearchQuery}
  />

  <FilterBar>
    <FilterDropdown
      label="Location"
      options={['Bangalore', 'Mumbai', 'Delhi NCR', 'Chennai', 'Hyderabad', 'Pune', 'Tier-2 Cities',
        ↵ 'Remote']}
      selected={filters.location}
      onChange={(val) => setFilter('location', val)}
    />

    <FilterDropdown
      label="Stage"
      options={['Idea', 'Pre-seed', 'Seed', 'Series A', 'Series B+', 'Growth']}
      selected={filters.stage}
      onChange={(val) => setFilter('stage', val)}
    />

    <FilterDropdown
      label="Sector"
      options={['Fintech', 'HealthTech', 'EdTech', 'SaaS', 'E-commerce', 'AI/ML', 'Consumer', 'B2B',
        ↵ 'Climate', 'Deeptech']}
      selected={filters.sector}
      onChange={(val) => setFilter('sector', val)}
    />

    <FilterDropdown
      label="Check Size"
      options={['$10K-$50K', '$50K-$250K', '$250K-$1M', '$1M-$5M', '$5M+']}
      selected={filters.checkSize}
      onChange={(val) => setFilter('checkSize', val)}
      showFor="investors"
    />
  </FilterBar>
</SearchPage>
```

```

/>

<Button variant="ghost" onClick={clearFilters}>
  Clear Filters
</Button>
</FilterBar>

<ResultsHeader>
  <Text>{results.length} results</Text>
  <SortDropdown
    options={[
      { value: 'relevance', label: 'Relevance' },
      { value: 'match_score', label: 'Match Score' },
      { value: 'newest', label: 'Newest' },
      { value: 'activity', label: 'Last Active' }
    ]}
    value={sortBy}
    onChange={setSortBy}
  />
</ResultsHeader>

<ResultsGrid>
  {results.map(profile => (
    <ProfileCard
      key={profile.id}
      profile={profile}
      matchScore={profile.matchScore}
      onView={() => viewProfile(profile.id)}
      onConnect={() => requestIntro(profile.id)}
    />
  ))}
</ResultsGrid>

<Pagination
  currentPage={page}
  totalPages={totalPages}
  onPageChange={setPage}
/>
</SearchPage>

```

Saved Searches:

```

<SavedSearches>
  <Text fontSize="sm" color="gray.500">Saved Searches</Text>
  {savedSearches.map(search => (
    <SavedSearchItem
      key={search.id}
      name={search.name}
      filters={search.filters}
      resultCount={search.resultCount}
      onClick={() => applySearch(search)}
    />
  ))}
</SavedSearches>

```



```
        onDelete={() => deleteSearch(search.id)}
      />
    )}
  <Button variant="ghost" size="sm" onClick={saveCurrentSearch}>
    + Save Current Search
  </Button>
</SavedSearches>
```

Search API:

```
GET
  ↪ /api/search?q={query}&location={location}&stage={stage}&sector={sector}&checkSize={checkSize}&sort={sort}&page={page}

Response:
{
  "success": true,
  "data": {
    "results": [...],
    "pagination": {
      "page": 1,
      "limit": 20,
      "total": 234,
      "totalPages": 12
    }
  }
}
```

10. Profile Verification System

Verification Tiers:

Tier	Badge	Requirements	Benefits
 Basic	Gray dot	Email verified	Standard matching
 Standard	Yellow badge	LinkedIn connected	+10% match priority
 Gold	Orange badge	Work email verified	+25% match priority, verified badge
 Platinum	Blue badge	ID + background check	+40% match priority, featured placement

Verification Flow:

```
<VerificationPage>
  <VerificationTier
    tier="basic"
    status="completed"
    completedAt="2026-03-20"
  />

  <VerificationTier
    tier="standard"
```

```

    status="available"
    onVerify={() => connectLinkedIn()}
  />

  <VerificationTier
    tier="gold"
    status="locked"
    requirements={['Complete Standard verification']}
  />

  <VerificationTier
    tier="platinum"
    status="locked"
    requirements={['Complete Gold verification']}
  />
</VerificationPage>

```

LinkedIn OAuth:

```

// Connect LinkedIn
const connectLinkedIn = async () => {
  const authUrl = `https://www.linkedin.com/oauth/v2/authorization?` +
    `response_type=code&` +
    `client_id=${LINKEDIN_CLIENT_ID}&` +
    `redirect_uri=${REDIRECT_URI}&` +
    `scope=r_liteprofile%20r_emailaddress`;

  window.location.href = authUrl;
};

// Callback handler
const handleLinkedInCallback = async (code) => {
  const response = await fetch('/api/auth/linkedin/callback', {
    method: 'POST',
    body: JSON.stringify({ code })
  });

  if (response.ok) {
    toast.success('LinkedIn connected successfully!');
    upgradeVerification('standard');
  }
};

```

Work Email Verification:

```

<WorkEmailVerification>
  <Input
    placeholder="you@company.com"
    value={workEmail}
    onChange={setWorkEmail}
  />
  <Button onClick={sendVerificationEmail}>

```

```

        Send Verification Code
    </Button>

    {codeSent && (
        <>
        <Input
            placeholder="Enter 6-digit code"
            value={verificationCode}
            onChange={setVerificationCode}
            maxLength={6}
        />
        <Button onClick={verifyCode}>
            Verify
        </Button>
        </>
    )}
</WorkEmailVerification>

```

11. Analytics Dashboard

Dashboard Layout:

```

<AnalyticsDashboard>
  <DashboardHeader
    title="Your Analytics"
    dateRange={dateRange}
    onDateRangeChange={setDateRange}
  />

  <StatsGrid columns={3}>
    <StatCard
      title="Profile Views"
      value={stats.profileViews}
      change={stats.profileViewsChange}
      icon={<EyeIcon />}
    />
    <StatCard
      title="Match Requests"
      value={stats.matchRequests}
      change={stats.matchRequestsChange}
      icon={<UserPlusIcon />}
    />
    <StatCard
      title="Match Rate"
      value={`${stats.matchRate}%`}
      change={stats.matchRateChange}
      icon={<ChartIcon />}
    />
  </StatsGrid>

  <ChartsSection>

```

```

<LineChart
  title="Profile Views Over Time"
  data={viewsOverTime}
  xAxis="date"
  yAxis="views"
/>

<BarChart
  title="Top Viewing Investors"
  data={topViewers}
  xAxis="name"
  yAxis="viewCount"
/>
</ChartsSection>

<RecentActivity>
  <Text fontSize="lg" fontWeight="bold">Recent Activity</Text>
  {activities.map(activity => (
    <ActivityItem
      key={activity.id}
      type={activity.type}
      user={activity.user}
      timestamp={activity.timestamp}
      description={activity.description}
    />
  ))}
</RecentActivity>
</AnalyticsDashboard>

```

Activity Types:

- profile_view - Someone viewed your profile
- match_request - Someone requested a match
- match_accepted - Your match request was accepted
- message_received - New message received
- intro_made - Introduction was facilitated

12. Referral Program

Referral System:

```

<ReferralPage>
  <ReferralHeader>
    <Text fontSize="2xl" fontWeight="bold">Invite Friends, Earn Rewards</Text>
    <Text>Share Cleo with your network and unlock premium features</Text>
  </ReferralHeader>

  <ReferralLink
    link={referralLink}
    onCopy={() => copyToClipboard(referralLink)}
  />
</ReferralPage>

```

```

/>

<ShareOptions>
  <ShareButton platform="twitter" />
  <ShareButton platform="linkedin" />
  <ShareButton platform="whatsapp" />
  <ShareButton platform="email" />
</ShareOptions>

<RewardsProgress>
  <Text fontSize="lg" fontWeight="bold">Your Progress</Text>
  <ProgressBar
    current={referralCount}
    milestones={[1, 5, 10, 25]}
  />

  <RewardsList>
    <RewardItem
      tier={1}
      requirement="1 referral"
      reward="1 month Premium"
      status={referralCount >= 1 ? 'claimed' : 'locked'}
    />
    <RewardItem
      tier={5}
      requirement="5 referrals"
      reward="6 months Premium"
      status={referralCount >= 5 ? 'claimed' : referralCount >= 1 ? 'unlocked' : 'locked'}
      progress={` ${referralCount}/5`}
    />
    <RewardItem
      tier={10}
      requirement="10 referrals"
      reward="Lifetime Premium"
      status={referralCount >= 10 ? 'claimed' : 'locked'}
      progress={` ${referralCount}/10`}
    />
    <RewardItem
      tier={25}
      requirement="25 referrals"
      reward="Ambassador Badge + Perks"
      status={referralCount >= 25 ? 'claimed' : 'locked'}
      progress={` ${referralCount}/25`}
    />
  </RewardsList>
</RewardsProgress>

<ReferralHistory>
  <Text fontSize="lg" fontWeight="bold">Referral History</Text>
  {referrals.map(ref => (
    <ReferralItem

```

```
      key={ref.id}
      email={ref.email}
      status={ref.status}
      joinedAt={ref.joinedAt}
      rewardClaimed={ref.rewardClaimed}
    />
  )}}
</ReferralHistory>
</ReferralPage>
```

TECHNICAL REQUIREMENTS

Tech Stack (Maintain Existing)

- Frontend: React + TypeScript + Vite
- Styling: Tailwind CSS
- UI Components: shadcn/ui
- Animation: Framer Motion
- Backend: Node.js + Express
- Database: PostgreSQL
- Auth: JWT + OAuth (Google, LinkedIn)
- Real-time: Socket.io
- File Storage: AWS S3 or Cloudflare R2

New Dependencies

```
# Animation
npm install framer-motion

# Icons
npm install lucide-react

# Forms & Validation
npm install react-hook-form zod @hookform/resolvers

# Toast Notifications
npm install sonner

# Charts
npm install recharts

# Date Handling
npm install date-fns

# WebSocket Client
npm install socket.io-client

# File Upload
npm install react-dropzone
```

Folder Structure

```
src/
├── components/
│   ├── ui/           # shadcn components
│   ├── layout/       # Layout components
│   ├── forms/        # Form components
│   ├── messaging/    # Messaging components
│   ├── search/       # Search components
│   └── analytics/    # Analytics components
├── hooks/
│   ├── useAuth.ts
│   ├── useMessaging.ts
│   ├── useSearch.ts
│   └── useAnalytics.ts
├── lib/
│   ├── utils.ts
│   ├── api.ts
│   ├── socket.ts
│   └── validation.ts
├── pages/
│   ├── Home.tsx
│   ├── Search.tsx
│   ├── Messages.tsx
│   ├── Analytics.tsx
│   ├── Profile.tsx
│   ├── Settings.tsx
│   └── Contact.tsx
├── types/
│   └── index.ts
└── context/
    ├── AuthContext.tsx
    └── SocketContext.tsx
```

API Endpoints to Add

```
// Search
GET    /api/search
GET    /api/search/saved
POST   /api/search/saved
DELETE /api/search/saved/:id

// Messaging
GET    /api/conversations
GET    /api/conversations/:id/messages
POST   /api/conversations/:id/messages
POST   /api/conversations/:id/read
DELETE /api/conversations/:id

// Verification
POST   /api/verification/linkedin
```

```
POST    /api/verification/email/send
POST    /api/verification/email/verify
GET     /api/verification/status
```

// Analytics

```
GET     /api/analytics/overview
GET     /api/analytics/profile-views
GET     /api/analytics/matches
GET     /api/analytics/activity
```

// Referrals

```
GET     /api/referrals
POST    /api/referrals/claim
GET     /api/referrals/stats
```

Database Migrations

-- Conversations

```
CREATE TABLE conversations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  participant_1_id UUID NOT NULL REFERENCES users(id),
  participant_2_id UUID NOT NULL REFERENCES users(id),
  created_at TIMESTAMP DEFAULT NOW(),
  last_message_at TIMESTAMP,
  UNIQUE(participant_1_id, participant_2_id)
);
```

-- Messages

```
CREATE TABLE messages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  conversation_id UUID NOT NULL REFERENCES conversations(id) ON DELETE CASCADE,
  sender_id UUID NOT NULL REFERENCES users(id),
  content TEXT NOT NULL,
  attachments JSONB DEFAULT '[]',
  read_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT NOW()
);
```

-- Saved Searches

```
CREATE TABLE saved_searches (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id),
  name VARCHAR(255) NOT NULL,
  filters JSONB NOT NULL,
  created_at TIMESTAMP DEFAULT NOW()
);
```

-- Referrals

```
CREATE TABLE referrals (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  referrer_id UUID NOT NULL REFERENCES users(id),
```



```

referred_email VARCHAR(255) NOT NULL,
referred_user_id UUID REFERENCES users(id),
status VARCHAR(50) DEFAULT 'pending',
reward_claimed BOOLEAN DEFAULT FALSE,
created_at TIMESTAMP DEFAULT NOW(),
converted_at TIMESTAMP
);

-- Verification
ALTER TABLE users ADD COLUMN verification_tier VARCHAR(50) DEFAULT 'basic';
ALTER TABLE users ADD COLUMN linkedin_id VARCHAR(255);
ALTER TABLE users ADD COLUMN work_email VARCHAR(255);
ALTER TABLE users ADD COLUMN work_email_verified_at TIMESTAMP;

-- Analytics tracking
CREATE TABLE profile_views (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  viewer_id UUID REFERENCES users(id),
  viewed_id UUID NOT NULL REFERENCES users(id),
  created_at TIMESTAMP DEFAULT NOW()
);

-- Indexes
CREATE INDEX idx_messages_conversation ON messages(conversation_id);
CREATE INDEX idx_messages_created_at ON messages(created_at DESC);
CREATE INDEX idx_conversations_last_message ON conversations(last_message_at DESC);
CREATE INDEX idx_profile_views_viewed ON profile_views(viewed_id);
CREATE INDEX idx_profile_views_created ON profile_views(created_at DESC);

```

DESIGN SPECIFICATIONS

Color Palette (Maintain)

- Primary: #10B981 (Emerald 500)
- Background: #0F0F1A (Dark)
- Surface: #1A1A2E (Card background)
- Text Primary: #FFFFFF
- Text Secondary: #A09FB5
- Border: #2A2A3E
- Error: #EF4444
- Success: #10B981
- Warning: #F59E0B

Typography

- Headings: Inter, 600-700 weight
- Body: Inter, 400 weight
- Code: JetBrains Mono

Spacing Scale

- xs: 4px
- sm: 8px
- md: 16px
- lg: 24px
- xl: 32px
- 2xl: 48px

Breakpoints

- Mobile: < 640px
- Tablet: 640px - 1024px
- Desktop: > 1024px

Animation Timing

- Fast: 150ms
- Normal: 300ms
- Slow: 500ms
- Easing: cubic-bezier(0.4, 0, 0.2, 1)

TESTING CHECKLIST

Before Deployment

- All critical bugs fixed
- Mobile navigation works on iOS/Android
- Form validation tested
- Toast notifications working
- Messaging system tested end-to-end
- Search filters working
- Analytics dashboard loading
- Accessibility audit passed (axe-core)
- Performance audit (Lighthouse 90+)
- Security headers verified

Browser Testing

- Chrome (latest)
 - Safari (latest)
 - Firefox (latest)
 - Edge (latest)
 - Chrome Mobile
 - Safari iOS
-

DEPLOYMENT NOTES

1. Run database migrations
2. Update environment variables
3. Deploy to staging first
4. Run smoke tests
5. Deploy to production
6. Monitor error rates
7. Check Core Web Vitals

Estimated Timeline: 4-6 weeks for all features **Priority Order:** Critical bugs → UI/UX → Messaging → Search → Verification → Analytics → Referrals